

Berlekamp Penaldo

Shahjalal University of Science and Technology

1	Geometry 3D	1
1.1	Point 3D	1
1.2	Plane	1
1.3	Line 3D	1
1.4	Plane Line	1
1.5	PolyHedron	2
1.6	Convex Hull 3d	2
1.7	Sphere	2

1 Geometry 3D

1.1 Point 3D

```
using P2 = Point<double>;
struct P3 {
    double x, y, z;
    explicit P3(double x = 0, double y = 0, double z = 0) : x(x), y(y), z(z) {}
    bool operator<(P3 p) const { return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(P3 p) const { return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P3 operator+(P3 p) const { return P3(x + p.x, y + p.y, z + p.z); }
    P3 operator-(P3 p) const { return P3(x - p.x, y - p.y, z - p.z); }
    P3 operator*(double d) const { return P3(x * d, y * d, z * d); }
    P3 operator/(double d) const { return P3(x / d, y / d, z / d); }
    double dot(P3 p) const { return x * p.x + y * p.y + z * p.z; }
    P3 cross(P3 p) const { return P3(y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x); }
    double dist2() const { return x * x + y * y + z * z; }
    double dist2(P3 p) const { return (*this - p).dist2(); }
    double dist() const { return sqrt(dist2()); }
    double dist(P3 p) const { return (*this - p).dist(); }
    P3 unit() const { return *this / dist(); }
    double angle(P3 p) const { return atan2(cross(p).dist(), dot(p)); }
    // (this*a).b
    double triple(P3 a, P3 b) const { return cross(a).dot(b); }
    // Smallest angle between this and p, ignoring direction, in [0, PI/2]
    double small_angle(P3 p) const { return acos(min(fabs(dot(p)) / dist(), 1.0)); }
    // Same direction, length r
    P3 truncate(double r) const { double k = dist(); return !sgn(k) ? *this : *this * (r / k); }
    // Rotate vector around unit axis 'u' by angle 'a' (Rodrigues' rotation formula)
    P3 rotate(P3 u, double a) const { u = u.unit(); return *this * cos(a) + u.cross(*this) * sin(a) + u * u.dot(*this) * (1 - cos(a)); }
    // Returns an arbitrary orthogonal unit vector to this
    P3 perp() const { return fabs(x) > EPS || fabs(y) > EPS ? P3(-y, x, 0).unit() : P3(0, -z, y).unit(); }
};
// orient' is >0 if s lies on lies on the same side of plane pqr as (q-p) x (r-p), <0 opposite, 0 on it.
```

```
// orient_by_normal: 2D-like orientation of p->q->r on the plane with normal n.
double orient(P3 p, P3 q, P3 r, P3 s) { return (q - p).cross(r - p).dot(s - p); }
double orient_by_normal(P3 p, P3 q, P3 r, P3 n) { return (q - p).cross(r - p).dot(n); }
```

1.2 Plane

```
// 3D Plane: normal n, offset d (n.dot(p) = d)
struct Plane {
    P3 n; double d;
    Plane(P3 n, double d) : n(n), d(d) {}
    Plane(P3 n, P3 p) : n(n), d(n.dot(p)) {}
    Plane(P3 p, P3 q, P3 r) : Plane((q - p).cross(r - p), p) {}
    double side(P3 p) const { return n.dot(p) - d; }
    double dist(P3 p) const { return fabs(side(p)) / n.dist(); }
    P3 proj(P3 p) const { return p - n * (side(p) / n.dist2()); }
    P3 reflect(P3 p) const { return p - n * (2 * side(p) / n.dist2()); }
    // [-PI, PI]
    double angle(Plane p) const { return n.angle(p.n); }
    // [0, PI/2]
    double small_angle(Plane p) const { return n.small_angle(p.n); }
    bool is_para(Plane p) const { return n.cross(p.n).dist() < EPS; }
    bool is_perp(Plane p) const { return sgn(n.dot(p.n)) == 0; }
    // Translate the plane by vector t
    Plane translate(P3 t) const { return { n, d + n.dot(t); }
    // Shift the plane along n by distance dis
    Plane shift_up(double dis) const { return { n, d + dis * n.dist(); }
    // Two distinct points lying on the plane (n must be non-zero)
    pair<P3, P3> two_pts() const { int sx = sgn(n.x), sy = sgn(n.y), sz = sgn(n.z); assert(sx || sy || sz); if (!sx && !sy) return {P3(1, 0, d / n.z), P3(0, 1, d / n.z)}; if (!sy && !sz) return {P3(d / n.x, 1, 0), P3(d / n.x, 0, 1)}; if (!sz && !sx) return {P3(1, d / n.y, 0), P3(0, d / n.y, 1)}; if (!sx) return {P3(1, d / n.y, 0), P3(0, 0, d / n.z)}; if (!sy) return {P3(0, 1, d / n.z), P3(d / n.x, 0, 0)}; if (!sz) return {P3(d / n.x, 0, 1), P3(0, d / n.y, 0)}; if (sgn(d)) return {P3(d / n.x, 0, 0), P3(0, d / n.y, 0)}; return {P3(n.y, -n.x, 0), P3(-n.y, n.x, 0)}; }
};
// Intersection line of two planes 3-point pairs.
pair<P3, P3> plane_inter(P3 a1, P3 b1, P3 c1, P3 a2, P3 b2, P3 c2) { auto n1 = (b1 - a1).cross(c1 - a1), n2 = (b2 - a2).cross(c2 - a2); auto d = n1.cross(n2); if (d.dist2() == 0) return {P3(), P3()}; auto o = (n2 * n1.dot(a1) - n1 * n2.dot(a2)).cross(d) / d.dist2(); return {o, o + d}; }
```

1.3 Line 3D

```
// 3d line parametric form: 'o + k * d'.
struct Line3D {
    P3 d, o;
    Line3D() {}
    Line3D(P3 p, P3 q) : d(q - p), o(p) {}
    Line3D(Plane p1, Plane p2) { d = p1.n.cross(p2.n); o = (p2.n * p1.d - p1.n * p2.d).cross(d) / d.dist2(); }
    double dist2(P3 p) const { return d.cross(p - o).dist2() / d.dist2(); }
    double dist(P3 p) const { return sqrt(dist2(p)); }
    P3 proj(P3 p) const { return o + d * (d.dot(p - o) / d.dist2()); }
    P3 inter(Plane p) const { return o - d * (p.side(o) / d.dot(p.n)); }
    // Orthogonal reflection of point p across the line
    P3 reflect(P3 p) const { return proj(p) * 2 - p; }
    // Compare points by their projection on the line (use as a sort comparator)
    bool cmp_proj(P3 p, P3 q) const { return d.dot(p) < d.dot(q); }
    // Small angle between the direction vectors of two lines
    double angle(Line3D l) const { return d.small_angle(l.d); }
    bool is_para(Line3D l) const { return d.cross(l.d).dist() < EPS; }
    bool is_perp(Line3D l) const { return sgn(d.dot(l.d)) == 0; }
    // Distance from 3D segment [p, q] to point x
    static double seg_dist(P3 p, P3 q, P3 x) { if ((q - p).dot(x - p) < 0) return x.dist(p); if ((p - q).dot(x - q) < 0) return x.dist(q); return (q - p).unit().cross(x - p).dist(); }
    // Check if point p lies on segment defined by [o, o + d]
    bool on_segment(P3 p) const { return dist(p) < EPS && (o - p).dot(o + d - p) <= EPS; }
};
// Line-line relations: min dist, closest point
double line_dist(Line3D l1, Line3D l2) { auto n = l1.d.cross(l2.d); if (n.dist2() == 0) return l1.dist(l2.o); return fabs((l2.o - l1.o).dot(n)) / n.dist(); }
P3 line_closest(Line3D l1, Line3D l2) { auto n2 = l2.d.cross(l1.d.cross(l2.d)); return l1.o + l1.d * ((l2.o - l1.o).dot(n2) / l1.d.dot(n2)); }
};
```

1.4 Plane Line

```
// Plane-line angles, relations, and perpendiculars through a point.
double angle(Plane p, Line3D l) { return PI / 2 - p.n.small_angle(l.d); }
bool is_para(Plane p, Line3D l) { return sgn(p.n.dot(l.d)) == 0; }
bool is_perp(Plane p, Line3D l) { return p.n.cross(l.d).dist() < EPS; }
Line3D perp_through(Plane p, P3 o) { return Line3D(o, o + p.n); }
Plane perp_through(Line3D l, P3 o) { return Plane(l.d, o); }
```

1.5 Polyhedron

```
// Converts 3D polygonal faces into
// flattened triangles: [TO_a, TO_b,
// TO_c, T1_a, ...]
vector<P3> flatten(const vector<vector<
P3>> &fs) {
    vector<P3> p;
    for (auto &f : fs)
        for (size_t j = 1; j + 1 < f.size();
            ++j)
            p.push_back(f[0]), p.push_back(f[j]), p.push_back(f[j + 1]);
    return p;
}

double poly_area(const vector<P3> &p) {
    double res = 0;
    for (size_t i = 0; i < p.size(); i += 3)
        res += (p[i + 1] - p[i]).cross(p[i + 2] - p[i]).dist();
    return res * 0.5;
}

double poly_volume(const vector<P3> &p) {
    P3 O(0, 0, 0);
    for (auto &pt : p) O = O + pt; O = O / p.size();
    double res = 0;
    for (size_t i = 0; i < p.size(); i += 3)
        res += fabs((p[i + 1] - p[i]).cross(p[i + 2] - p[i]).dot(p[i] - O));
    return res / 6.0;
}

P3 poly_centroid(const vector<P3> &p) {
    P3 O(0, 0, 0);
    for (auto &pt : p) O = O + pt; O = O / p.size();
    P3 c(O, 0, 0);
    double tv = 0;
    for (size_t i = 0; i < p.size(); i += 3) {
        double v = fabs((p[i + 1] - p[i]).cross(p[i + 2] - p[i]).dot(p[i] - O));
        c = c + (O + p[i] + p[i + 1] + p[i + 2]) * (v / 4.0);
        tv += v;
    }
    return c / tv;
}

bool contains(const vector<P3> &p, P3 pt) {
    P3 O(0, 0, 0);
    for (auto &pt_i : p) O = O + pt_i; O = O / p.size();
    for (size_t i = 0; i < p.size(); i += 3) {
        P3 n = (p[i + 1] - p[i]).cross(p[i + 2] - p[i]);
        if (n.dot(p[i] - O) < 0) n = n * -1.0; // Auto-flip to outward normal
        if (n.dot(pt - p[i]) > EPS) return false;
    }
    return true;
}

// Distance from point pt to the plane
// of face starting at index i
double pt_face_dist(const vector<P3> &p, size_t i, P3 pt) {
    P3 n = (p[i + 1] - p[i]).cross(p[i + 2] - p[i]);
    return fabs(n.dot(pt - p[i])) / n.dist();
}

// Returns total count of surface
// triangles
size_t num_tri(const vector<P3> &p) {
    return p.size() / 3;
}

// Returns count of unique surface
// polygons by merging adjacent
// coplanar triangles
int num_poly(const vector<P3> &p) {
    int res = 0;
    for (size_t i = 0; i < p.size(); i += 3) {
        bool ok = true;
        for (size_t j = 0; j < i; j += 3) {
            P3 n = (p[j + 1] - p[j]).cross(p[j + 2] - p[j]);
            if (fabs(n.dot(p[i] - p[j])) < EPS
                && fabs(n.dot(p[i + 1] - p[j])) < EPS
                && fabs(n.dot(p[i + 2] - p[j])) < EPS) {
                ok = false; break;
            }
        }
        res += ok;
    }
    return res;
}

// 0(N^2) Incremental 3D Convex Hull
// builder (outputs flattened triangle
// vector)
struct CH3D {
    struct face { int a, b, c; bool ok; };
    size_t n, num;
    vector<P3> P;
    vector<face> F;
    vector<vector<int>> g;
    double dblcmp(P3 p, face f) { return ((P[f.b] - P[f.a]).cross(P[f.c] - P[f.a])).dot(p - P[f.a]); }
    void deal(int p, int a, int b) {
        int f = g[a][b];
        if (F[f].ok) {
            if (dblcmp(P[p], F[f]) > EPS) dfs(p, f);
        } else {
            g[p][b] = g[a][p] = g[b][a] = num;
            F[num++] = {b, a, p, true};
        }
    }
    void dfs(int p, int now) {
        F[now].ok = false;
        deal(p, F[now].b, F[now].a);
        deal(p, F[now].c, F[now].b);
        deal(p, F[now].a, F[now].c);
    }
    vector<P3> build(vector<P3> p) {
        P = p; n = P.size(); num = 0;
        if (n < 4) return {};
        F.resize(8 * n + 1); g.assign(n + 1, vector<int>(n + 1, 0));
        bool ok = false;
        for (size_t i = 1; i < n; ++i) if (P[0].dist(P[i]) > EPS) { swap(P[1], P[i]); ok = true; break; }
        if (!ok) return {}; ok = false;
        for (size_t i = 2; i < n; ++i) if ((P[0] - P[1]).cross(P[1] - P[i]).dist() > EPS) { swap(P[2], P[i]); ok = true; break; }
        if (!ok) return {}; ok = false;
        for (size_t i = 3; i < n; ++i) if (fabs((P[0] - P[1]).cross(P[1] - P[2])).dot(P[0] - P[i]) > EPS) { swap(P[3], P[i]); ok = true; break; }
        if (!ok) return {};
        for (int i = 0; i < 4; ++i) {
            face add = {(i + 1) % 4, (i + 2) % 4, (i + 3) % 4, true};
            if (dblcmp(P[i], add) > 0) swap(add.b, add.c);
            g[add.a][add.b] = g[add.b][add.c] = g[add.c][add.a] = num;
            F[num++] = add;
        }
        for (size_t i = 4; i < n; ++i)
            for (size_t j = 0; j < num; ++j)
                if (F[j].ok && dblcmp(P[i], F[j]) > EPS) { dfs(i, j); break; }
        vector<P3> res;
        for (size_t i = 0; i < num; ++i) if (F[i].ok) {
            res.push_back(P[F[i].a]); res.push_back(P[F[i].b]); res.push_back(P[F[i].c]);
        }
        return res;
    }
};
```

1.6 Convex Hull 3d

```
// 0(N^2) Incremental 3D Convex Hull
// builder (outputs flattened triangle
// vector)
struct CH3D {
    struct face { int a, b, c; bool ok; };
    size_t n, num;
    vector<P3> P;
    vector<face> F;
    vector<vector<int>> g;
    double dblcmp(P3 p, face f) { return ((P[f.b] - P[f.a]).cross(P[f.c] - P[f.a])).dot(p - P[f.a]); }
    void deal(int p, int a, int b) {
        int f = g[a][b];
        if (F[f].ok) {
            if (dblcmp(P[p], F[f]) > EPS) dfs(p, f);
        } else {
            g[p][b] = g[a][p] = g[b][a] = num;
            F[num++] = {b, a, p, true};
        }
    }
    void dfs(int p, int now) {
        F[now].ok = false;
        deal(p, F[now].b, F[now].a);
        deal(p, F[now].c, F[now].b);
        deal(p, F[now].a, F[now].c);
    }
    vector<P3> build(vector<P3> p) {
        P = p; n = P.size(); num = 0;
        if (n < 4) return {};
        F.resize(8 * n + 1); g.assign(n + 1, vector<int>(n + 1, 0));
        bool ok = false;
        for (size_t i = 1; i < n; ++i) if (P[0].dist(P[i]) > EPS) { swap(P[1], P[i]); ok = true; break; }
        if (!ok) return {}; ok = false;
        for (size_t i = 2; i < n; ++i) if ((P[0] - P[1]).cross(P[1] - P[i]).dist() > EPS) { swap(P[2], P[i]); ok = true; break; }
        if (!ok) return {}; ok = false;
        for (size_t i = 3; i < n; ++i) if (fabs((P[0] - P[1]).cross(P[1] - P[2])).dot(P[0] - P[i]) > EPS) { swap(P[3], P[i]); ok = true; break; }
        if (!ok) return {};
        for (int i = 0; i < 4; ++i) {
            face add = {(i + 1) % 4, (i + 2) % 4, (i + 3) % 4, true};
            if (dblcmp(P[i], add) > 0) swap(add.b, add.c);
            g[add.a][add.b] = g[add.b][add.c] = g[add.c][add.a] = num;
            F[num++] = add;
        }
        for (size_t i = 4; i < n; ++i)
            for (size_t j = 0; j < num; ++j)
                if (F[j].ok && dblcmp(P[i], F[j]) > EPS) { dfs(i, j); break; }
        vector<P3> res;
        for (size_t i = 0; i < num; ++i) if (F[i].ok) {
            res.push_back(P[F[i].a]); res.push_back(P[F[i].b]); res.push_back(P[F[i].c]);
        }
        return res;
    }
};
```

1.7 Sphere

```
// Spherical coordinates, distances,
// and sphere-sphere intersections.
P3 sph_pt(double r, double lat, double lon) {
    lat *= PI / 180; lon *= PI / 180;
    return P3(r * cos(lat) * cos(lon), r * cos(lat) * sin(lon), r * sin(lat));
}

double great_circle_dist(P3 o, double r, P3 a, P3 b) { return r * (a - o).angle(b - o); }

struct Sphere {
    P3 c; double r;
    double volume() const { return 4.0 / 3.0 * PI * r * r * r; }
    double surface_area() const { return 4 * PI * r * r; }
    // Line-Sphere Intersection Points
    vector<P3> inter(Line3D l) const {
        P3 proj = l.proj(c);
        double d2 = c.dist2(proj);
        if (d2 > r * r + EPS) return {};
        double h = sqrt(max(0.0, r * r - d2));
        P3 dir = l.d.unit();
        if (h < EPS) return {proj};
        return {proj - dir * h, proj + dir * h};
    }
    // if this point is inside the sphere
    bool contains(P3 p) const { return sgn(p.dist(c) - r) <= 0; }
    // Length of the tangent segment from
    // an external point p (-1 if p is
    // inside)
    double tangent_len(P3 p) const {
        double d = p.dist(c);
        return sgn(d - r) < 0 ? -1 : sqrt(max(0.0, d * d - r * r));
    }
    // Polar angle of the cap of tangency
    // points from p (-1 if inside, -2 if
    // on the surface)
    double tangent_angle(P3 p) const {
        double d = p.dist(c);
        if (sgn(d - r) < 0) return -1;
        if (sgn(d - r) == 0) return -2;
        return acos(r / d);
    }
};

// sphere-sphere intersection
pair<double, double> sphere_inter(Sphere s1, Sphere s2) {
    double d = s1.c.dist(s2.c);
    if (d >= s1.r + s2.r) return {0, 0};
    double R = max(s1.r, s2.r), r = min(s1.r, s2.r);
    double x = (R * R - r * r + d * d) / (2 * d);
    double h1 = R - x, h2 = (R + r - d) - h1;
    double vol = PI * h1 * h1 * (3 * R - h1) / 3.0 + PI * h2 * h2 * (3 * r - h2) / 3.0;
    double area = 2 * PI * R * h1 + 2 * PI * r * h2;
    return {area, vol};
}

// Radius of the circle in which two
// intersecting spheres meet
double sphere_inter_radius(Sphere s1, Sphere s2) {
    double d = s1.c.dist(s2.c), R = max(s1.r, s2.r), r = min(s1.r, s2.r);
    if (sgn(d - R - r) >= 0) return 0;
    double w = d * d - r * r + R * R;
    return sqrt(max(0.0, 4 * d * d * R * R - w * w)) / (2 * d);
}

// Sphere through four non-coplanar
// points
Sphere circum_sphere(P3 a, P3 b, P3 c, P3 d) {
    assert(sgn(Plane(a, b, c).side(d)) != 0);
}
```

```

Plane u(a - b, (a + b) / 2), v(b - c,
    (b + c) / 2), w(c - d, (c + d) / 2)
;
Line3D l1(u, v), l2(v, w);
P3 o = line_closest(l1, l2);
return {o, o.dist(a)};
}
// Center of the circle through three
// non-collinear coplanar points.
P3 circle_center(P3 a, P3 b, P3 c) {
    P3 v1 = b - a, v2 = c - a;
    double v11 = v1.dot(v1), v22 = v2.dot(
        v2), v12 = v1.dot(v2);
    double base = 0.5 / (v11 * v22 - v12 *
        v12);
    return a + v1 * (base * v22 * (v11 -
        v12)) + v2 * (base * v11 * (v22 -
        v12));
}
// approximate min enclosing sphere
Sphere min_sphere(vector<P3> p) {
    P3 c(0, 0, 0); int n = p.size(), pos =
        0;
    for (auto &i : p) c = c + i;
    c = c / n;
    double ratio = 0.1;
    for (int it = 0; it < 100000; it++) {
        pos = 0;
        for (int i = 1; i < n; i++) if (c.
            dist2(p[i]) > c.dist2(p[pos])) pos
            = i;
        c = c + (p[pos] - c) * ratio;
        ratio *= 0.998;
    }
    return {c, c.dist(p[pos])};
}

```